Batch 23 — CRM + Customer Retention Pack

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:24

Batch 23 — CRM + Customer Retention Pack

Meat Lovers CIMS powered by YohPal

23A.

database/migrations/045_customer_visit_history.sql

```
CREATE TABLE customer_visit_history (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    customer_id BIGINT NOT NULL,
    order_id BIGINT,

    visit_date DATE NOT NULL,
    visit_source ENUM(
        'WALK_IN',
        'WEBSITE',
        'DELIVERY',
        'CATERING',
        'PHONE',
        'WHATSAPP'
    ) DEFAULT 'WALK_IN',

    total_spent DECIMAL(12,2) DEFAULT 0,

    FOREIGN KEY (customer_id) REFERENCES customers(id),
    FOREIGN KEY (order_id) REFERENCES orders(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

23B.

database/migrations/046_customer_follow_ups.sql

```
CREATE TABLE customer_follow_ups (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    customer_id BIGINT,
    website_lead_id BIGINT,
    feedback_id BIGINT,

    follow_up_type ENUM(
        'WEBSITE_LEAD',
        'FEEDBACK',
        'ABANDONED_LEAD',
        'BIRTHDAY_REMINDER',
        'ANNIVERSARY_REMINDER',
        'LOYALTY_REWARD',
        'MANAGER_CALL'
    ) NOT NULL,

    follow_up_status ENUM(
        'PENDING',
        'CONTACTED',
        'CONVERTED',
        'CLOSED'
    ) DEFAULT 'PENDING',

    assigned_to BIGINT,

    notes TEXT,
    due_date DATE,

    FOREIGN KEY (customer_id) REFERENCES customers(id),
    FOREIGN KEY (website_lead_id) REFERENCES website_leads(id),
```

```
    FOREIGN KEY (feedback_id) REFERENCES customer_feedback(id),
    FOREIGN KEY (assigned_to) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

23C.

database/migrations/047_customer_profiles_extension.sql

```
ALTER TABLE customers
ADD COLUMN birth_date DATE NULL AFTER customer_type,
ADD COLUMN anniversary_date DATE NULL AFTER birth_date,
ADD COLUMN segment ENUM(
    'NEW',
    'REGULAR',
    'VIP',
    'DORMANT',
    'CORPORATE',
    'DELIVERY_ONLY'
) DEFAULT 'NEW' AFTER anniversary_date,
ADD COLUMN last_visit_date DATE NULL AFTER loyalty_points,
ADD COLUMN total_visits INT DEFAULT 0 AFTER last_visit_date,
ADD COLUMN lifetime_value DECIMAL(12,2) DEFAULT 0 AFTER total_visits;
```

23D.

database/migrations/048_loyalty_transactions.sql

```
CREATE TABLE loyalty_transactions (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    customer_id BIGINT NOT NULL,

    transaction_type ENUM(
        'EARNED',
        'REDEEMED',
        'ADJUSTED'
    ) NOT NULL,

    points DECIMAL(12,2) NOT NULL,

    reference_type ENUM(
        'ORDER',
        'MANUAL',
        'PROMOTION'
    ) DEFAULT 'ORDER',

    reference_id BIGINT,

    notes TEXT,

    FOREIGN KEY (customer_id) REFERENCES customers(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

23E. Update

backend/routes/api.php

```
$router->get('/api/crm/customer-segments', [CustomersController::class, 'customerSegments']);
$router->get('/api/crm/visit-history', [CustomersController::class, 'visitHistory']);
$router->post('/api/crm/visit-history', [CustomersController::class, 'recordVisit']);

$router->get('/api/crm/loyalty-transactions', [CustomersController::class, 'loyaltyTransactions']);
$router->post('/api/crm/loyalty-transactions', [CustomersController::class, 'recordLoyaltyTransaction']);

$router->get('/api/crm/follow-ups', [CustomersController::class, 'followUps']);
$router->post('/api/crm/follow-ups', [CustomersController::class, 'createFollowUp']);
$router->post('/api/crm/follow-ups/{id}/status', [CustomersController::class, 'updateFollowUpStatus']);

$router->post('/api/crm/generate-abandoned-lead-followups', [CustomersController::class, '
generateAbandonedLeadFollowUps']);
$router->post('/api/crm/generate-reminder-placeholders', [CustomersController::class,
'generateReminderPlaceholders']);

$router->get('/api/crm/repeat-customer-dashboard', [CustomersController::class, 'repeatCustomerDashboard']);
```

23F. Update

backend/app/Controllers/CustomersController.php

Add imports:

```
use Support\Auth;
use Support\AuditLogger;
```

Add methods inside the class:

```
public function customerSegments(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CRM']);

    $stmt = Database::connection()->query(
        "SELECT
            segment,
            COUNT(*) AS customer_count,
            COALESCE(SUM(lifetime_value), 0) AS lifetime_value
         FROM customers
         GROUP BY segment
         ORDER BY customer_count DESC"
    );

    Response::success(['customer_segments' => $stmt->fetchAll()]);
}

public function visitHistory(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CRM']);

    $stmt = Database::connection()->query(
        "SELECT
            cvh.*,
            c.full_name,
            c.phone
         FROM customer_visit_history cvh
         JOIN customers c ON c.id = cvh.customer_id
         ORDER BY cvh.visit_date DESC, cvh.created_at DESC"
    );

    Response::success(['visit_history' => $stmt->fetchAll()]);
}

public function recordVisit(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CRM', 'CASHIER']);
    $data = Request::body();
    $db = Database::connection();

    $db->beginTransaction();

    $stmt = $db->prepare(
        'INSERT INTO customer_visit_history
        (customer_id, order_id, visit_date, visit_source, total_spent)
        VALUES (:customer_id, :order_id, :visit_date, :visit_source, :total_spent)'
    );

    $stmt->execute([
        'customer_id' => $data['customer_id'],
        'order_id' => $data['order_id'] ?? null,
        'visit_date' => $data['visit_date'],
        'visit_source' => $data['visit_source'] ?? 'WALK_IN',
        'total_spent' => $data['total_spent'] ?? 0,
    ]);

    $update = $db->prepare(
        'UPDATE customers
         SET total_visits = total_visits + 1,
             last_visit_date = :visit_date,
             lifetime_value = lifetime_value + :total_spent,
             segment = CASE
                WHEN lifetime_value + :total_spent >= 50000 THEN "VIP"
                WHEN total_visits + 1 >= 3 THEN "REGULAR"
                ELSE segment
             END
         WHERE id = :customer_id'
    );

    $update->execute([
        'visit_date' => $data['visit_date'],
        'total_spent' => $data['total_spent'] ?? 0,
        'customer_id' => $data['customer_id'],
```

```
        ]);

        AuditLogger::log(
            (int) $user['id'],
            'CRM',
            'RECORD_CUSTOMER_VISIT',
            (int) $data['customer_id'],
            null,
            $data
        );

        $db->commit();

        Response::success([], 'Customer visit recorded');
    }

    public function loyaltyTransactions(): void
    {
        Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CRM']);

        $stmt = Database::connection()->query(
            "SELECT
                lt.*,
                c.full_name,
                c.phone
             FROM loyalty_transactions lt
             JOIN customers c ON c.id = lt.customer_id
             ORDER BY lt.created_at DESC"
        );

        Response::success(['loyalty_transactions' => $stmt->fetchAll()]);
    }

    public function recordLoyaltyTransaction(): void
    {
        $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CRM', 'CASHIER']);
        $data = Request::body();
        $db = Database::connection();

        $db->beginTransaction();

        $stmt = $db->prepare(
            'INSERT INTO loyalty_transactions
            (customer_id, transaction_type, points, reference_type, reference_id, notes)
            VALUES (:customer_id, :transaction_type, :points, :reference_type, :reference_id, :notes)'
        );

        $stmt->execute([
            'customer_id' => $data['customer_id'],
            'transaction_type' => $data['transaction_type'],
            'points' => $data['points'],
            'reference_type' => $data['reference_type'] ?? 'MANUAL',
            'reference_id' => $data['reference_id'] ?? null,
            'notes' => $data['notes'] ?? null,
        ]);

        $pointsImpact = $data['transaction_type'] === 'REDEEMED'
            ? -1 * (float) $data['points']
            : (float) $data['points'];

        $update = $db->prepare(
            'UPDATE customers
             SET loyalty_points = loyalty_points + :points
             WHERE id = :customer_id'
        );

        $update->execute([
            'points' => $pointsImpact,
            'customer_id' => $data['customer_id'],
        ]);

        AuditLogger::log(
            (int) $user['id'],
            'CRM',
            'RECORD_LOYALTY_TRANSACTION',
            (int) $data['customer_id'],
            null,
            $data
        );

        $db->commit();

        Response::success([], 'Loyalty transaction recorded');
    }

    public function followUps(): void
    {
```

```
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CRM']);

    $stmt = Database::connection()->query(
        "SELECT
            cf.*,
            c.full_name AS customer_name,
            c.phone AS customer_phone,
            u.full_name AS assigned_to_name
         FROM customer_follow_ups cf
         LEFT JOIN customers c ON c.id = cf.customer_id
         LEFT JOIN users u ON u.id = cf.assigned_to
         ORDER BY cf.created_at DESC"
    );

    Response::success(['follow_ups' => $stmt->fetchAll()]);
}

public function createFollowUp(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CRM']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO customer_follow_ups
        (customer_id, website_lead_id, feedback_id, follow_up_type, follow_up_status, assigned_to, notes,
due_date)
        VALUES (:customer_id, :website_lead_id, :feedback_id, :type, "PENDING", :assigned_to, :notes,
:due_date)'
    );

    $stmt->execute([
        'customer_id' => $data['customer_id'] ?? null,
        'website_lead_id' => $data['website_lead_id'] ?? null,
        'feedback_id' => $data['feedback_id'] ?? null,
        'type' => $data['follow_up_type'],
        'assigned_to' => $data['assigned_to'] ?? $user['id'],
        'notes' => $data['notes'] ?? null,
        'due_date' => $data['due_date'] ?? null,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'CRM',
        'CREATE_CUSTOMER_FOLLOW_UP',
        null,
        null,
        $data
    );

    Response::success([], 'Customer follow-up created');
}

public function updateFollowUpStatus(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CRM']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'UPDATE customer_follow_ups
         SET follow_up_status = :status,
             notes = CONCAT(COALESCE(notes, ""), "\n", :notes)
         WHERE id = :id'
    );

    $stmt->execute([
        'status' => $data['follow_up_status'],
        'notes' => $data['notes'] ?? '',
        'id' => $id,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'CRM',
        'UPDATE_FOLLOW_UP_STATUS',
        $id,
        null,
        $data
    );

    Response::success([], 'Follow-up status updated');
}

public function generateAbandonedLeadFollowUps(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CRM']);
    $db = Database::connection();
```

```
    $stmt = $db->query(
        "SELECT wl.*
         FROM website_leads wl
         LEFT JOIN customer_follow_ups cf ON cf.website_lead_id = wl.id
         WHERE wl.lead_status = 'NEW'
         AND cf.id IS NULL"
    );

    $leads = $stmt->fetchAll();
    $created = 0;

    foreach ($leads as $lead) {
        $insert = $db->prepare(
            'INSERT INTO customer_follow_ups
            (website_lead_id, follow_up_type, follow_up_status, assigned_to, notes, due_date)
            VALUES (:lead_id, "ABANDONED_LEAD", "PENDING", :assigned_to, :notes, CURDATE())'
        );

        $insert->execute([
            'lead_id' => $lead['id'],
            'assigned_to' => $user['id'],
            'notes' => 'Website lead has not yet been contacted',
        ]);

        $created++;
    }

    AuditLogger::log(
        (int) $user['id'],
        'CRM',
        'GENERATE_ABANDONED_LEAD_FOLLOWUPS',
        null,
        null,
        ['created' => $created]
    );

    Response::success(['created' => $created], 'Abandoned lead follow-ups generated');
}

public function generateReminderPlaceholders(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CRM']);
    $db = Database::connection();

    $stmt = $db->query(
        "SELECT id, birth_date, anniversary_date
         FROM customers
         WHERE birth_date IS NOT NULL
         OR anniversary_date IS NOT NULL"
    );

    $customers = $stmt->fetchAll();
    $created = 0;

    foreach ($customers as $customer) {
        if (!empty($customer['birth_date'])) {
            $insert = $db->prepare(
                'INSERT INTO customer_follow_ups
                (customer_id, follow_up_type, follow_up_status, assigned_to, notes, due_date)
                VALUES (:customer_id, "BIRTHDAY_REMINDER", "PENDING", :assigned_to, "Birthday reminder
placeholder", :due_date)'
            );

            $insert->execute([
                'customer_id' => $customer['id'],
                'assigned_to' => $user['id'],
                'due_date' => $customer['birth_date'],
            ]);

            $created++;
        }

        if (!empty($customer['anniversary_date'])) {
            $insert = $db->prepare(
                'INSERT INTO customer_follow_ups
                (customer_id, follow_up_type, follow_up_status, assigned_to, notes, due_date)
                VALUES (:customer_id, "ANNIVERSARY_REMINDER", "PENDING", :assigned_to, "Anniversary reminder
placeholder", :due_date)'
            );

            $insert->execute([
                'customer_id' => $customer['id'],
                'assigned_to' => $user['id'],
                'due_date' => $customer['anniversary_date'],
            ]);

            $created++;
```

```
        }
    }

    AuditLogger::log(
        (int) $user['id'],
        'CRM',
        'GENERATE_REMINDER_PLACEHOLDERS',
        null,
        null,
        ['created' => $created]
    );

    Response::success(['created' => $created], 'Reminder placeholders generated');
}

public function repeatCustomerDashboard(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CRM']);

    $db = Database::connection();

    $summary = $db->query(
        "SELECT
            COUNT(*) AS total_customers,
            SUM(CASE WHEN total_visits >= 2 THEN 1 ELSE 0 END) AS repeat_customers,
            SUM(CASE WHEN segment = 'VIP' THEN 1 ELSE 0 END) AS vip_customers,
            SUM(loyalty_points) AS total_loyalty_points,
            SUM(lifetime_value) AS total_lifetime_value
         FROM customers"
    )->fetch();

    $topCustomers = $db->query(
        "SELECT
            id,
            full_name,
            phone,
            segment,
            total_visits,
            loyalty_points,
            lifetime_value,
            last_visit_date
         FROM customers
         ORDER BY lifetime_value DESC, total_visits DESC
         LIMIT 20"
    )->fetchAll();

    Response::success([
        'summary' => $summary,
        'top_customers' => $topCustomers,
    ]);
}
```

23G. Update

apps/admin-web/src/features/operations/api/operationsApi.js

```
customerSegments: async () => {
  const { data } = await apiClient.get('/crm/customer-segments')
  return data.data.customer_segments
},

visitHistory: async () => {
  const { data } = await apiClient.get('/crm/visit-history')
  return data.data.visit_history
},

recordVisit: async (payload) => {
  const { data } = await apiClient.post('/crm/visit-history', payload)
  return data
},

loyaltyTransactions: async () => {
  const { data } = await apiClient.get('/crm/loyalty-transactions')
  return data.data.loyalty_transactions
},

recordLoyaltyTransaction: async (payload) => {
  const { data } = await apiClient.post('/crm/loyalty-transactions', payload)
  return data
},

followUps: async () => {
  const { data } = await apiClient.get('/crm/follow-ups')
  return data.data.follow_ups
```

```
},

createFollowUp: async (payload) => {
  const { data } = await apiClient.post('/crm/follow-ups', payload)
  return data
},

updateFollowUpStatus: async (id, payload) => {
  const { data } = await apiClient.post(`/crm/follow-ups/${id}/status`, payload)
  return data
},

generateAbandonedLeadFollowUps: async () => {
  const { data } = await apiClient.post('/crm/generate-abandoned-lead-followups', {})
  return data
},

generateReminderPlaceholders: async () => {
  const { data } = await apiClient.post('/crm/generate-reminder-placeholders', {})
  return data
},

repeatCustomerDashboard: async () => {
  const { data } = await apiClient.get('/crm/repeat-customer-dashboard')
  return data.data
},
```

23H.

apps/admin-web/src/features/crm/pages/CustomerSegmentsPage.jsx

```
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function CustomerSegmentsPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['customer-segments'],
    queryFn: operationsApi.customerSegments,
  })

  return (
    <Page title="Customer Segments" subtitle="Customer segmentation by value and loyalty">
      <Card>
        {isLoading ? <LoadingBlock label="Loading customer segments..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}
        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'segment', label: 'Segment' },
              { key: 'customer_count', label: 'Customers' },
              { key: 'lifetime_value', label: 'Lifetime Value' },
            ]}
            rows={data || []}
          />
        ) : null}
      </Card>
    </Page>
  )
}
```

23I.

apps/admin-web/src/features/crm/pages/VisitHistoryPage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function VisitHistoryPage() {
  return (
    <OperationalFormPage
      title="Customer Visit History"
      subtitle="Record and monitor customer visits"
      queryKey="visit-history"
      queryFn={operationsApi.visitHistory}
      mutationFn={operationsApi.recordVisit}
```

```
      initialForm={{
        customer_id: '',
        order_id: '',
        visit_date: '',
        visit_source: 'WALK_IN',
        total_spent: '',
      }}
      buildPayload={(form) => ({
        customer_id: Number(form.customer_id),
        order_id: form.order_id ? Number(form.order_id) : null,
        visit_date: form.visit_date,
        visit_source: form.visit_source,
        total_spent: Number(form.total_spent || 0),
      })}
      fields={[
        { name: 'customer_id', label: 'Customer ID', type: 'number' },
        { name: 'order_id', label: 'Order ID optional', type: 'number' },
        { name: 'visit_date', label: 'Visit Date', type: 'date' },
        {
          name: 'visit_source',
          label: 'Visit Source',
          type: 'select',
          options: [
            { value: 'WALK_IN', label: 'Walk In' },
            { value: 'WEBSITE', label: 'Website' },
            { value: 'DELIVERY', label: 'Delivery' },
            { value: 'CATERING', label: 'Catering' },
            { value: 'PHONE', label: 'Phone' },
            { value: 'WHATSAPP', label: 'WhatsApp' },
          ],
        },
        { name: 'total_spent', label: 'Total Spent', type: 'number' },
      ]}
      columns={[
        { key: 'full_name', label: 'Customer' },
        { key: 'phone', label: 'Phone' },
        { key: 'visit_date', label: 'Visit Date' },
        { key: 'visit_source', label: 'Source' },
        { key: 'total_spent', label: 'Spent' },
      ]}
      submitLabel="Record Visit"
    />
  )
}
```

23J.

apps/admin-web/src/features/crm/pages/LoyaltyTransactionsPage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function LoyaltyTransactionsPage() {
  return (
    <OperationalFormPage
      title="Loyalty Transactions"
      subtitle="Earn, redeem, or adjust customer loyalty points"
      queryKey="loyalty-transactions"
      queryFn={operationsApi.loyaltyTransactions}
      mutationFn={operationsApi.recordLoyaltyTransaction}
      initialForm={{
        customer_id: '',
        transaction_type: 'EARNED',
        points: '',
        reference_type: 'MANUAL',
        reference_id: '',
        notes: '',
      }}
      buildPayload={(form) => ({
        customer_id: Number(form.customer_id),
        transaction_type: form.transaction_type,
        points: Number(form.points),
        reference_type: form.reference_type,
        reference_id: form.reference_id ? Number(form.reference_id) : null,
        notes: form.notes,
      })}
      fields={[
        { name: 'customer_id', label: 'Customer ID', type: 'number' },
        {
          name: 'transaction_type',
          label: 'Transaction Type',
          type: 'select',
          options: [
```

```
            { value: 'EARNED', label: 'Earned' },
            { value: 'REDEEMED', label: 'Redeemed' },
            { value: 'ADJUSTED', label: 'Adjusted' },
          ],
        },
        { name: 'points', label: 'Points', type: 'number' },
        {
          name: 'reference_type',
          label: 'Reference Type',
          type: 'select',
          options: [
            { value: 'ORDER', label: 'Order' },
            { value: 'MANUAL', label: 'Manual' },
            { value: 'PROMOTION', label: 'Promotion' },
          ],
        },
        { name: 'reference_id', label: 'Reference ID optional', type: 'number' },
        { name: 'notes', label: 'Notes', type: 'textarea' },
      ]}
      columns={[
        { key: 'full_name', label: 'Customer' },
        { key: 'phone', label: 'Phone' },
        { key: 'transaction_type', label: 'Type' },
        { key: 'points', label: 'Points' },
        { key: 'reference_type', label: 'Reference Type' },
        { key: 'created_at', label: 'Date' },
      ]}
      submitLabel="Record Loyalty"
    />
  )
}
```

23K.

apps/admin-web/src/features/crm/pages/FollowUpsPage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function FollowUpsPage() {
  return (
    <OperationalFormPage
      title="Customer Follow-Ups"
      subtitle="Manage feedback, abandoned leads, reminders, and loyalty follow-ups"
      queryKey="customer-follow-ups"
      queryFn={operationsApi.followUps}
      mutationFn={operationsApi.createFollowUp}
      initialForm={{
        customer_id: '',
        website_lead_id: '',
        feedback_id: '',
        follow_up_type: 'MANAGER_CALL',
        assigned_to: '',
        due_date: '',
        notes: '',
      }}
      buildPayload={(form) => ({
        customer_id: form.customer_id ? Number(form.customer_id) : null,
        website_lead_id: form.website_lead_id ? Number(form.website_lead_id) : null,
        feedback_id: form.feedback_id ? Number(form.feedback_id) : null,
        follow_up_type: form.follow_up_type,
        assigned_to: form.assigned_to ? Number(form.assigned_to) : null,
        due_date: form.due_date,
        notes: form.notes,
      })}
      fields={[
        { name: 'customer_id', label: 'Customer ID optional', type: 'number' },
        { name: 'website_lead_id', label: 'Website Lead ID optional', type: 'number' },
        { name: 'feedback_id', label: 'Feedback ID optional', type: 'number' },
        {
          name: 'follow_up_type',
          label: 'Follow-Up Type',
          type: 'select',
          options: [
            { value: 'WEBSITE_LEAD', label: 'Website Lead' },
            { value: 'FEEDBACK', label: 'Feedback' },
            { value: 'ABANDONED_LEAD', label: 'Abandoned Lead' },
            { value: 'BIRTHDAY_REMINDER', label: 'Birthday Reminder' },
            { value: 'ANNIVERSARY_REMINDER', label: 'Anniversary Reminder' },
            { value: 'LOYALTY_REWARD', label: 'Loyalty Reward' },
            { value: 'MANAGER_CALL', label: 'Manager Call' },
          ],
        },
```

```
        { name: 'assigned_to', label: 'Assigned Staff ID optional', type: 'number' },
        { name: 'due_date', label: 'Due Date', type: 'date' },
        { name: 'notes', label: 'Notes', type: 'textarea' },
      ]}
      columns={[
        { key: 'customer_name', label: 'Customer' },
        { key: 'customer_phone', label: 'Phone' },
        { key: 'follow_up_type', label: 'Type' },
        { key: 'follow_up_status', label: 'Status' },
        { key: 'assigned_to_name', label: 'Assigned To' },
        { key: 'due_date', label: 'Due Date' },
      ]}
      submitLabel="Create Follow-Up"
    />
  )
}
```

23L.

apps/admin-web/src/features/crm/pages/RepeatCustomerDashboardPage.jsx

```
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function RepeatCustomerDashboardPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['repeat-customer-dashboard'],
    queryFn: operationsApi.repeatCustomerDashboard,
  })

  return (
    <Page title="Repeat Customer Dashboard" subtitle="Customer retention, loyalty, and lifetime value">
      {isLoading ? <LoadingBlock label="Loading repeat customer dashboard..." /> : null}
      {error ? <StatusMessage error={error.message} /> : null}

      {data ? (
        <>
          <div style={{ display: 'grid', gridTemplateColumns: 'repeat(5, minmax(0, 1fr))', gap: 16 }}>
            <Card title="Total Customers"><strong>{data.summary.total_customers}</strong></Card>
            <Card title="Repeat Customers"><strong>{data.summary.repeat_customers}</strong></Card>
            <Card title="VIP Customers"><strong>{data.summary.vip_customers}</strong></Card>
            <Card title="Loyalty Points"><strong>{data.summary.total_loyalty_points}</strong></Card>
            <Card title="Lifetime Value"><strong>{data.summary.total_lifetime_value}</strong></Card>
          </div>

          <Card title="Top Customers">
            <DataTable
              columns={[
                { key: 'full_name', label: 'Customer' },
                { key: 'phone', label: 'Phone' },
                { key: 'segment', label: 'Segment' },
                { key: 'total_visits', label: 'Visits' },
                { key: 'loyalty_points', label: 'Points' },
                { key: 'lifetime_value', label: 'Lifetime Value' },
                { key: 'last_visit_date', label: 'Last Visit' },
              ]}
              rows={data.top_customers || []}
            />
          </Card>
        </>
      ) : null}
    </Page>
  )
}
```

23M.

apps/admin-web/src/features/crm/pages/CRMActionToolsPage.jsx

```
import React, { useState } from 'react'
import { useMutation } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../operations/api/operationsApi'
```

```
export default function CRMActionToolsPage() {
  const [success, setSuccess] = useState('')

  const abandonedMutation = useMutation({
    mutationFn: operationsApi.generateAbandonedLeadFollowUps,
    onSuccess: (result) => setSuccess(`Created ${result.data.created} abandoned lead follow-ups`),
  })

  const reminderMutation = useMutation({
    mutationFn: operationsApi.generateReminderPlaceholders,
    onSuccess: (result) => setSuccess(`Created ${result.data.created} reminder placeholders`),
  })

  return (
    <Page title="CRM Action Tools" subtitle="Generate follow-ups and reminder placeholders">
      <StatusMessage
        success={success}
        error={abandonedMutation.error?.message || reminderMutation.error?.message}
      />

      <div style={{ display: 'grid', gap: 16 }}>
        <Card title="Abandoned Website Leads">
          <p>Create follow-ups for website leads still marked as NEW.</p>
          <Button onClick={() => abandonedMutation.mutate()}>
            Generate Abandoned Lead Follow-Ups
          </Button>
        </Card>

        <Card title="Birthday / Anniversary Placeholder">
          <p>Create reminder placeholder follow-ups for customers with saved dates.</p>
          <Button onClick={() => reminderMutation.mutate()}>
            Generate Reminder Placeholders
          </Button>
        </Card>
      </div>
    </Page>
  )
}
```

23N. Update

apps/admin-web/src/app/routes.jsx

Add imports:

```
import CustomerSegmentsPage from '../features/crm/pages/CustomerSegmentsPage'
import VisitHistoryPage from '../features/crm/pages/VisitHistoryPage'
import LoyaltyTransactionsPage from '../features/crm/pages/LoyaltyTransactionsPage'
import FollowUpsPage from '../features/crm/pages/FollowUpsPage'
import RepeatCustomerDashboardPage from '../features/crm/pages/RepeatCustomerDashboardPage'
import CRMActionToolsPage from '../features/crm/pages/CRMActionToolsPage'
```

Add routes:

```
<Route path="/crm/segments" element={<CustomerSegmentsPage />} />
<Route path="/crm/visit-history" element={<VisitHistoryPage />} />
<Route path="/crm/loyalty" element={<LoyaltyTransactionsPage />} />
<Route path="/crm/follow-ups" element={<FollowUpsPage />} />
<Route path="/crm/repeat-dashboard" element={<RepeatCustomerDashboardPage />} />
<Route path="/crm/action-tools" element={<CRMActionToolsPage />} />
```

23O. Update

apps/admin-web/src/components/common/SidebarNav.jsx

```
['CRM Segments', '/crm/segments'],
['Visit History', '/crm/visit-history'],
['Loyalty Points', '/crm/loyalty'],
['CRM Follow-Ups', '/crm/follow-ups'],
['Repeat Customers', '/crm/repeat-dashboard'],
['CRM Action Tools', '/crm/action-tools'],
```

23P.

docs/CRM_CUSTOMER_RETENTION_PACK.md

```
# Batch 23 — CRM + Customer Retention Pack

## System
```

```
Meat Lovers CIMS powered by YohPal

## Purpose

This batch turns customer data into a retention and repeat-sales system.

## Features Added

1. Customer segmentation
2. Visit history
3. Loyalty points
4. Feedback follow-up
5. Birthday reminder placeholder
6. Anniversary reminder placeholder
7. Abandoned website lead follow-up
8. Repeat-customer dashboard

## Customer Segments

Supported segments:

- NEW
- REGULAR
- VIP
- DORMANT
- CORPORATE
- DELIVERY_ONLY

## Visit History

Each visit records:

- customer
- order optional
- visit date
- visit source
- total spent

## Loyalty Points

Loyalty transactions support:

- earned points
- redeemed points
- manual adjustments

## Follow-Ups

Follow-ups support:

- website leads
- abandoned leads
- feedback
- birthday reminders
- anniversary reminders
- loyalty rewards
- manager calls

## Repeat Customer Dashboard

Dashboard shows:

- total customers
- repeat customers
- VIP customers
- loyalty points issued
- customer lifetime value
- top customers

## Business Value

This batch helps Meat Lovers:

- increase repeat sales
- follow up abandoned website leads
- reward loyal customers
- identify VIP customers
- follow up bad feedback
- improve customer retention
```

Batch 23 Outcome

CRM + customer retention now includes:

✅ customer segmentation
✅ visit history
✅ loyalty points
✅ feedback follow-up
✅ birthday/anniversary reminder placeholder
✅ abandoned website lead follow-up
✅ repeat-customer dashboard

Next Smart Move

Build **Batch 24 — Asset Inventory + Maintenance Control Pack**, including asset assignment, maintenance schedules, repair logs, damage reports, asset write-off approval placeholder, and asset lifecycle dashboard.